Google Git

Sign in Auto

android / platform / system / core / refs/heads/main / . / **fastboot**

tree: c66af46cc55138dd6ee70b51cbeb4501b83d23ec [search]

- device/
- fuzzer/
- fuzzy_fastboot/
- testdata/
- .clang-format
  ⇨ ../.clang-format-4
- Android.bp
- bootimg_utils.cpp
- bootimg_utils.h
- constants.h
- fastboot.bash
- fastboot.cpp
- fastboot.h
- fastboot_driver.cpp
- fastboot_driver.h
- fastboot_driver_interface.h
- fastboot_driver_mock.h
- fastboot_driver_test.cpp
- fastboot_integration_test.xml
- fastboot_test.cpp
- filesystem.cpp
- filesystem.h
- fs.cpp
- fs.h
- LICENSE
- main.cpp
- mock_transport.h
- OWNERS
- README.md
- result.h
- socket.cpp
- socket.h
- socket_mock.cpp
- socket_mock.h

- socket_test.cpp
- storage.cpp
- storage.h
- super_flash_helper.cpp
- super_flash_helper.h
- super_flash_helper_test.cpp
- task.cpp
- task.h
- task_test.cpp
- tcp.cpp
- tcp.h
- tcp_test.cpp
- test_fastboot.py
- TEST_MAPPING
- transport.h
- udp.cpp
- udp.h
- udp_test.cpp
- usb.h
- usb_linux.cpp
- usb_osx.cpp
- usb_windows.cpp
- util.cpp
- util.h
- vendor_boot_img_utils.cpp
- vendor_boot_img_utils.h
- vendor_boot_img_utils_test.cpp

---

fastboot/README.md

Fastboot

The fastboot protocol is a mechanism for communicating with bootloaders over USB or ethernet. It is designed to be very straightforward to implement, to allow it to be used across a wide range of devices and from hosts running Linux, macOS, or Windows.

Basic Requirements

- USB
  - Two bulk endpoints (in, out) are required
  - Max packet size must be 64 bytes for full-speed, 512 bytes for high-speed and 1024 bytes for Super Speed USB.

    - The protocol is entirely host-driven and synchronous (unlike the multi-channel, bi-directional, asynchronous ADB protocol)

- TCP or UDP
    - Device must be reachable via IP.
    - Device will act as the server, fastboot will be the client.
    - Fastboot data is wrapped in a simple protocol; see below for details.

Transport and Framing

1. Host sends a command, which is an ascii string in a single packet no greater than 4096 bytes.
2. Client response with a single packet no greater than 256 bytes. The first four bytes of the response are “OKAY”, “FAIL”, “DATA”, “INFO” or “TEXT”. Additional bytes may contain an (ascii) informative message.

   a. INFO -> the remaining 252 bytes are an informative message (providing progress or diagnostic messages). They should be displayed and then step #2 repeats. The print format is: "(bootloader) " + InfoMessagePayload + ‘\n’

   b. TEXT -> the remaining 252 bytes are arbitrary. They should be displayed and then step #2 repeats. It differs from info in that no formatting is applied. The payload is printed as-is with no newline at the end. Payload is expected to be NULL terminated.

   c. FAIL -> the requested command failed. The remaining 252 bytes of the response (if present) provide a textual failure message to present to the user. Stop.

   d. OKAY -> the requested command completed successfully. Go to #5

   e. DATA -> the requested command is ready for the data phase. A DATA response packet will be 12 bytes long, in the form of DATA00000000 where the 8 digit hexadecimal number represents the total data size to transfer.
3. Data phase. Depending on the command, the host or client will send the indicated amount of data. Short packets are always acceptable and zero-length packets are ignored. This phase continues until the client has sent or received the number of bytes indicated in the “DATA” response above.
4. Client responds with a single packet no greater than 256 bytes. The first four bytes of the response are “OKAY”, “FAIL”, “INFO” or “TEXT”. Similar to #2:

   a. INFO -> display the formatted remaining 252 bytes and return to #4

   b. TEXT -> display the unformatted remaining 252 bytes and return to #4

   c. FAIL -> display the remaining 252 bytes (if present) as a failure reason and consider the command failed. Stop.

   d. OKAY -> success. Go to #5
5. Success. Stop.

Example Session

```
Host:    "getvar:version"        request version variable

Client:  "OKAY0.4"               return version "0.4"

Host:    "getvar:nonexistant"    request some undefined variable

Client:  "FAILUnknown variable"  getvar failure; see getvar details below

Host:    "download:00001234"     request to send 0x1234 bytes of data

Client:  "DATA00001234"          ready to accept data

Host:    < 0x1234 bytes >        send data

Client:  "OKAY"                  success

Host:    "flash:bootloader"      request to flash the data to the bootloader

Client:  "INFOerasing flash"     indicate status / progress
         "INFOwriting flash"
         "OKAY"                  indicate success

Host:    "powerdown"             send a command

Client:  "FAILunknown command"   indicate failure
```

Command Reference

- Command parameters are indicated by printf-style escape sequences.
- Commands are ascii strings and sent without the quotes (which are for illustration only here) and without a trailing 0 byte.
- Commands that begin with a lowercase letter are reserved for this specification. OEM-specific commands should not begin with a lowercase letter, to prevent incompatibilities with future specs.

The various currently defined commands are:

```
getvar:%s          Read a config/version variable from the bootloader.
                   The variable contents will be returned after the
                   OKAY response. If the variable is unknown, the bootloader
                   should return a FAIL response, optionally with an error
                   message.
```

```
                    Previous versions of this document indicated that getvar
                    should return an empty OKAY response for unknown
                    variables, so older devices might exhibit this behavior,
                    but new implementations should return FAIL instead.

download:%08x       Write data to memory which will be later used
                    by "boot", "ramdisk", "flash", etc.  The client
                    will reply with "DATA%08x" if it has enough
                    space in RAM or "FAIL" if not.  The size of
                    the download is remembered.

upload              Read data from memory which was staged by the last
                    command, e.g. an oem command.  The client will reply
                    with "DATA%08x" if it is ready to send %08x bytes of
                    data.  If no data was staged in the last command,
                    the client must reply with "FAIL".  After the client
                    successfully sends %08x bytes, the client shall send
                    a single packet starting with "OKAY".  Clients
                    should not support "upload" unless it supports an
                    oem command that requires "upload" capabilities.

flash:%s            Write the previously downloaded image to the
                    named partition (if possible).

erase:%s            Erase the indicated partition (clear to 0xFFs)

boot                The previously downloaded data is a boot.img
                    and should be booted according to the normal
                    procedure for a boot.img

continue            Continue booting as normal (if possible)

reboot              Reboot the device.

reboot-bootloader
                    Reboot back into the bootloader.
                    Useful for upgrade processes that require upgrading
                    the bootloader and then upgrading other partitions
                    using the new bootloader.
```

Flashing Logic

Fastboot binary will follow directions listed out fastboot-info.txt build artifact for fastboot flashall && fastboot update comamnds. This build artifact will live inside of ANDROID_PRODUCT_OUT && target_files_package && updatepackage.

The currently defined commands are:

```
flash %s           Flash a given partition. Optional arguments include
                   --slot-other, {filename_path}, --apply-vbmeta

reboot %s          Reboot to either bootloader or fastbootd

update-super       Updates the super partition

if-wipe            Conditionally run some other functionality if
                   wipe is specified

erase %s           Erase a given partition (can only be used in conjunction)
                   with if-wipe -> eg. if-wipe erase cache
```

Flashing Optimization:

```
After generating the list of tasks to execute, Fastboot will try and
optimize the flashing of the dynamic partitions by constructing an
optimized flash super task. Fastboot will explicitly pattern match the
following commands and try and concatenate it into this task. (doing so
will allow us to avoid the reboot into userspace fastbootd which takes
significant time)

//Optimizable Block
reboot fastboot
update-super                         ---> generate optimized flash super task
$FOR EACH {dynamic partition}
    flash {dynamic partition}
```

Client Variables

The “getvar:%s” command is used to read client variables which represent various information about the device and the software on it.

The various currently defined names are:

```
version             Version of FastBoot protocol supported.
                    It should be "0.4" for this document.

version-bootloader  Version string for the Bootloader.

version-baseband    Version string of the Baseband Software

product             Name of the product

serialno            Product serial number
```

```
secure              If the value is "yes", this is a secure
                    bootloader requiring a signature before
                    it will install or boot images.

is-userspace        If the value is "yes", the device is running
                    fastbootd. Otherwise, it is running fastboot
                    in the bootloader.
```

Names starting with a lowercase character are reserved by this specification. OEM-specific names should not start with lowercase characters.

Logical Partitions

There are a number of commands to interact with logical partitions:

```
update-super:%s:%s  Write the previously downloaded image to a super
                    partition. Unlike the "flash" command, this has
                    special rules. The image must have been created by
                    the lpmake command, and must not be a sparse image.
                    If the last argument is "wipe", then all existing
                    logical partitions are deleted. If no final argument
                    is specified, the partition tables are merged. Any
                    partition in the new image that does not exist in the
                    old image is created with a zero size.

                    In all cases, this will cause the temporary "scratch"
                    partition to be deleted if it exists.

create-logical-partition:%s:%d
                    Create a logical partition with the given name and
                    size, in the super partition.

delete-logical-partition:%s
                    Delete a logical partition with the given name.

resize-logical-partition:%s:%d
                    Change the size of the named logical partition.
```

In addition, there is a variable to test whether a partition is logical:

```
is-logical:%s       If the value is "yes", the partition is logical.
                    Otherwise the partition is physical.
```

TCP Protocol v1

The TCP protocol is designed to be a simple way to use the fastboot protocol over ethernet if USB is not available.

The device will open a TCP server on port 5554 and wait for a fastboot client to connect.

Handshake

Upon connecting, both sides will send a 4-byte handshake message to ensure they are speaking the same protocol. This consists of the ASCII characters “FB” followed by a 2-digit base-10 ASCII version number. For example, the version 1 handshake message will be [FB01].

If either side detects a malformed handshake, it should disconnect.

The protocol version to use must be the minimum of the versions sent by each side; if either side cannot speak this protocol version, it should disconnect.

Fastboot Data

Once the handshake is complete, fastboot data will be sent as follows:

```
[data_size][data]
```

Where data_size is an unsigned 8-byte big-endian binary value, and data is the fastboot packet. The 8-byte length is intended to provide future-proofing even though currently fastboot packets have a 4-byte maximum length.

Example

In this example the fastboot host queries the device for two variables, “version” and “none”.

```
Host    <connect to the device on port 5555>
Host    FB01
Device  FB01
Host    [0x00][0x00][0x00][0x00][0x00][0x00][0x00][0x0E]getvar:version
Device  [0x00][0x00][0x00][0x00][0x00][0x00][0x00][0x07]OKAY0.4
Host    [0x00][0x00][0x00][0x00][0x00][0x00][0x00][0x0B]getvar:none
Device  [0x00][0x00][0x00][0x00][0x00][0x00][0x00][0x14]FAILUnknown variable
Host    <disconnect>
```

UDP Protocol v1

The UDP protocol is more complex than TCP since we must implement reliability to ensure no packets are lost, but the general concept of wrapping the fastboot protocol is the same.

Overview:

1. As with TCP, the device will listen on UDP port 5554.

2. Maximum UDP packet size is negotiated during initialization.
3. The host drives all communication; the device may only send a packet as a response to a host packet.
4. If the host does not receive a response in 500ms it will re-transmit.

UDP Packet format

```
+----------+----+-------+-------+--------------------+
| Byte #   | 0  |   1   | 2 - 3 |  4+                |
+----------+----+-------+-------+--------------------+
| Contents | ID | Flags | Seq # | Data               |
+----------+----+-------+-------+--------------------+

ID      Packet ID:
          0x00: Error.
          0x01: Query.
          0x02: Initialization.
          0x03: Fastboot.

        Packet types are described in more detail below.

Flags   Packet flags: 0 0 0 0 0 0 0 C
          C=1 indicates a continuation packet; the data is too large and will
              continue in the next packet.

          Remaining bits are reserved for future use and must be set to 0.

Seq #   2-byte packet sequence number (big-endian). The host will increment
        this by 1 with each new packet, and the device must provide the
        corresponding sequence number in the response packets.

Data    Packet data, not present in all packets.
```

Packet Types

```
Query
      The host sends a query packet once on startup to sync with the device.
      The host will not know the current sequence number, so the device must
      respond to all query packets regardless of sequence number.

      The response data field should contain a 2-byte big-endian value
      giving the next expected sequence number.

Init
      The host sends an init packet once the query response is returned. The
      device must abort any in-progress operation and prepare for a new
```

```
      fastboot session. This message is meant to allow recovery if a
      previous session failed, e.g. due to network error or user Ctrl+C.

      The data field contains two big-endian 2-byte values, a protocol
      version and the max UDP packet size (including the 4-byte header).
      Both the host and device will send these values, and in each case
      the minimum of the sent values must be used.

Fastboot
      These packets wrap the fastboot protocol. To write, the host will
      send a packet with fastboot data, and the device will reply with an
      empty packet as an ACK. To read, the host will send an empty packet,
      and the device will reply with fastboot data. The device may not give
      any data in the ACK packet.

Error
      The device may respond to any packet with an error packet to indicate
      a UDP protocol error. The data field should contain an ASCII string
      describing the error. This is the only case where a device is allowed
      to return a packet ID other than the one sent by the host.
```

Packet Size

The maximum packet size is negotiated by the host and device in the Init packet. Devices must support at least 512-byte packets, but packet size has a direct correlation with download speed, so devices are strongly suggested to support at least 1024-byte packets. On a local network with 0.5ms round-trip time this will provide transfer rates of ~2MB/s. Over WiFi it will likely be significantly less.

Query and Initialization packets, which are sent before size negotiation is complete, must always be 512 bytes or less.

Packet Re-Transmission

The host will re-transmit any packet that does not receive a response. The requirement of exactly one device response packet per host packet is how we achieve reliability and in-order delivery of packets.

For simplicity of implementation, there is no windowing of multiple unacknowledged packets in this version of the protocol. The host will continue to send the same packet until a response is received. Windowing functionality may be implemented in future versions if necessary to increase performance.

The first Query packet will only be attempted a small number of times, but subsequent packets will attempt to retransmit for at least 1 minute before giving up. This means a device may safely ignore host UDP packets for up to 1 minute during long operations, e.g. writing to flash.

Continuation Packets

Any packet may set the continuation flag to indicate that the data is incomplete. Large data such as downloading an image may require many continuation packets. The receiver should respond to a continuation packet with an empty packet to acknowledge receipt. See examples below.

Summary

The host starts with a Query packet, then an Initialization packet, after which only Fastboot packets are sent. Fastboot packets may contain data from the host for writes, or from the device for reads, but not both.

Given a next expected sequence number S and a received packet P, the device behavior should be:

```
if P is a Query packet:
  * respond with a Query packet with S in the data field
else if P has sequence == S:
  * process P and take any required action
  * create a response packet R with the same ID and sequence as P, containing
    any response data required.
  * transmit R and save it in case of re-transmission
  * increment S
else if P has sequence == S - 1:
  * re-transmit the saved response packet R from above
else:
  * ignore the packet
```

Examples

In the examples below, S indicates the starting client sequence number.

```
Host                                    Client
=======================================================================
[Initialization, S = 0x55AA]
[Host: version 1, 2048-byte packets. Client: version 2, 1024-byte packets.]
[Resulting values to use: version = 1, max packet size = 1024]
ID   Flag SeqH SeqL Data                ID   Flag SeqH SeqL Data
-----------------------------------------------------------------------
0x01 0x00 0x00 0x00
                                        0x01 0x00 0x00 0x00 0x55 0xAA
0x02 0x00 0x55 0xAA 0x00 0x01 0x08 0x00
                                        0x02 0x00 0x55 0xAA 0x00 0x02 0x04 0x00


-----------------------------------------------------------------------
[fastboot "getvar" commands, S = 0x0001]
ID    Flags SeqH  SeqL  Data            ID    Flags SeqH  SeqL  Data
-----------------------------------------------------------------------
0x03  0x00  0x00  0x01  getvar:version
                                        0x03  0x00  0x00  0x01
```

```
0x03  0x00  0x00  0x02
                                        0x03  0x00  0x00  0x02  OKAY0.4
0x03  0x00  0x00  0x03  getvar:none
                                        0x03  0x00  0x00  0x03
0x03  0x00  0x00  0x04
                                        0x03  0x00  0x00  0x04  FAILUnknown var

------------------------------------------------------------------------
[fastboot "INFO" responses, S = 0x0000]
ID    Flags SeqH  SeqL  Data            ID    Flags SeqH  SeqL  Data
------------------------------------------------------------------------
0x03  0x00  0x00  0x00  <command>
                                        0x03  0x00  0x00  0x00
0x03  0x00  0x00  0x01
                                        0x03  0x00  0x00  0x01  INFOWait1
0x03  0x00  0x00  0x02
                                        0x03  0x00  0x00  0x02  INFOWait2
0x03  0x00  0x00  0x03
                                        0x03  0x00  0x00  0x03  OKAY

------------------------------------------------------------------------
[Chunking 2100 bytes of data, max packet size = 1024, S = 0xFFFF]
ID   Flag SeqH SeqL Data                ID   Flag SeqH SeqL Data
------------------------------------------------------------------------
0x03 0x00 0xFF 0xFF download:0000834
                                        0x03 0x00 0xFF 0xFF
0x03 0x00 0x00 0x00
                                        0x03 0x00 0x00 0x00 DATA0000834
0x03 0x01 0x00 0x01 <1020 bytes>
                                        0x03 0x00 0x00 0x01
0x03 0x01 0x00 0x02 <1020 bytes>
                                        0x03 0x00 0x00 0x02
0x03 0x00 0x00 0x03 <60 bytes>
                                        0x03 0x00 0x00 0x03
0x03 0x00 0x00 0x04
                                        0x03 0x00 0x00 0x04 OKAY

------------------------------------------------------------------------
[Unknown ID error, S = 0x0000]
ID    Flags SeqH  SeqL  Data            ID    Flags SeqH  SeqL  Data
------------------------------------------------------------------------
0x10  0x00  0x00  0x00
                                        0x00  0x00  0x00  0x00  <error message>

------------------------------------------------------------------------
[Host packet loss and retransmission, S = 0x0000]
ID    Flags SeqH  SeqL  Data            ID    Flags SeqH  SeqL  Data
```

```
------------------------------------------------------------------------
0x03  0x00  0x00  0x00  getvar:version [lost]
0x03  0x00  0x00  0x00  getvar:version [lost]
0x03  0x00  0x00  0x00  getvar:version
                                      0x03  0x00  0x00  0x00
0x03  0x00  0x00  0x01
                                      0x03  0x00  0x00  0x01  OKAY0.4

------------------------------------------------------------------------
[Client packet loss and retransmission, S = 0x0000]
ID    Flags SeqH  SeqL  Data            ID    Flags SeqH  SeqL  Data
------------------------------------------------------------------------
0x03  0x00  0x00  0x00  getvar:version
                                      0x03  0x00  0x00  0x00 [lost]
0x03  0x00  0x00  0x00  getvar:version
                                      0x03  0x00  0x00  0x00 [lost]
0x03  0x00  0x00  0x00  getvar:version
                                      0x03  0x00  0x00  0x00
0x03  0x00  0x00  0x01
                                      0x03  0x00  0x00  0x01  OKAY0.4

------------------------------------------------------------------------
[Host packet delayed, S = 0x0000]
ID    Flags SeqH  SeqL  Data            ID    Flags SeqH  SeqL  Data
------------------------------------------------------------------------
0x03  0x00  0x00  0x00  getvar:version [delayed]
0x03  0x00  0x00  0x00  getvar:version
                                      0x03  0x00  0x00  0x00
0x03  0x00  0x00  0x01
                                      0x03  0x00  0x00  0x01  OKAY0.4
0x03  0x00  0x00  0x00  getvar:version [arrives late with old seq#, is ignored]
```

Powered by Gitiles| Privacy| Terms

txt json